Main ↔ BallMotion

03 / Mapa de integración y trazabilidad

Cada fila vincula un uso concreto del módulo matemático en main con su contrato. Los enlaces azules abren el código local o el PDF asociado. Mantener los tres PDF en la misma carpeta para los enlaces entre manuales; la apertura depende del visor.

Uso en main	Elemento de BallMotion / efecto
src/main.cpp:14	Motion: instancia global; orientation persiste entre muestras. fullSpeed usa su valor por defecto.
src/main.cpp:15	Vec: bias, sum y muestras. Construcción por componentes, escala, suma/resta y norm().
src/main.cpp:16	Output: almacenamiento de la última orden; {} anula ambos ejes.
src/main.cpp:22	Vec y pi: convierte raw a g/rad/s con constructor y multiplicación escalar.
src/main.cpp:33	Motion::reset: identidad al iniciar calibración. También se usa al esperar sensor, conectar, entrar en demo y completar calibración.
src/main.cpp:120	Quat::inverse / rotate: demo convierte giro fijo y gravedad a ejes del sensor simulado.
src/main.cpp:121	Motion::update: demo; cuarto argumento omitido, movimiento habilitado por defecto.
src/main.cpp:128	Vec::norm / operadores: valida quietud, suma gyro y obtiene bias=sum/300.
src/main.cpp:142	Motion::update: real; gyro-bias, accel, dt y habilitación solo en Ready.
src/main.cpp:147	Motion::orientation / Quat: lectura de w,x,y,z para publicar K2.

Uso indirecto y extensión sin uso

cross(), multiplicación de Quat y normalize() son internos a la matemática, no llamadas directas de main. onVerticalRotation() no se invoca. Ambos archivos usan ControllerConfig.h; no es una dependencia del transporte.

Flujos y límites de integración

03 / Contratos entre módulos

Datos reales

```
MPU6050 -> readSample() -> gyro [rad/s], accel [g]
        -> gyro - bias -> Motion::update(..., dt, enabled)
        -> Output + orientation -> Serial K2 -> receptor futuro
```

Punto de ejecución	Entradas / resultados
AwaitingAlignment	update(..., false): orientación provisional activa; orden cero. Bias previo o cero inicial. Confirmación/bypass decide cuándo calibrar.
Calibrating	No llama update(). Acumula muestras aceptadas; al completar, guarda bias, reinicia orientación y cambia a Ready.
Ready	update(..., true): orientación y orden reales. Dirección en marco del jugador; intensidad normalizada.
Demo	Entradas sintéticas; update() con movimiento habilitado. No lee sensor ni aplica bias real.
Waiting / fallo	No actualiza orientación. main reinicia a identidad y anula salida; el estado indica que no es pose medida.

Propiedad de las decisiones

main: conexión, temporización, estado, bias, confirmación y publicación. BallMotion: inclinación, integración, cambio de ejes, zona muerta y normalización. Unity futuro: recepción, UI de alineación y aplicación del movimiento del juego.

Frontera futura con Unity

La orientación enviada es relativa y no prueba por sí sola el frente físico. El jugador alinea marcas de frente/arriba y confirma. La futura recepción deberá establecer alineadoEnUnity=true en AwaitingAlignment; main consume la confirmación al calibrar. Con bypass=true no espera. No hay implementación Bluetooth ni de Unity en esta entrega.

El receptor debe adaptar ejes/orden del cuaternión, ignorar diagnósticos #, distinguir demo y estados inválidos, y anular movimiento ante datos caducados. No asumir continuidad visual al reset: el cuaternión pasa a identidad.

Próximos pasos: validación de integración

1. Demo sin sensor: en PlatformIO, entorno nodemcu-32s, ejecutar Build, Upload y Monitor (COM4, 115200); enviar d. Revisar juntos K2/demo: orden cero en reposo y giro vertical, signos correctos y mayor intensidad en avance rápido. La orientación debe cambiar al girar. c sale de demo. Esta sesión con el usuario sigue pendiente.

2. Hardware real: validar montaje, calibración, ruido, zona muerta y tiempos de muestreo. Al implementar Bluetooth, medir también transmisión y recepción; no confundir su latencia con el intervalo local del sensor.

Fuentes de esta documentación: código local de main.cpp, BallMotion.h, ControllerConfig.h, platformio.ini y test/motion_host.cpp. Describe el estado implementado, no una API futura prometida.